18 B 樹 解答

18-E01 節點容量

推導

非根 key 下限 t-1=3，上限 2t-1=7。7 個 key 時子指標上限為 8。

結論與檢核

葉有 6 個 key 在 3 到 7 之間，合法。除空樹外根至少 1 key，但根可有較少量鍵。

18-E02 根分裂

推導

中位 key 20 升上新根，左子 [10]，右子 [30]。插入 25 時，25>20 往右，25<30 放在右子，得右子 [25,30]。

結論與檢核

新根 [20]；插入完成後左子 [10] 含 1 鍵、右子 [25,30] 含 2 鍵，都在 t-1=1 到 2t-1=3 的合法範圍。分裂剛完成時兩子才都恰有一鍵。

18-E03 I/O 高度

推導

用邊數定義高度h。最大滿樹容量$(2t)^{h+1}-1$，故$n<200^4-1=1{,}599{,}999{,}999$可由h=3容納，但h=2最大僅7,999,999，不足十億。最小合法$n\geq 2t^h-1$；h=5至少$2\cdot 100^5-1>10^9$，h=4可行。因此高度為3或4，搜尋至葉需4或5個節點頁，根快取時實際頁讀再少一。

結論與檢核

在指定 t=100,n=10^9 的容量限制下，最壞搜尋路徑 4 至 5 個節點，依實際填滿率而定。這是節點容量與 I/O 模型界限，非硬碟延遲實測。

18-E04 借鍵時不能漏掉子樹

推導

左child僅1鍵=t−1，右child2鍵可借。父20下移成左child末鍵，右child首鍵30上升成根，右child剩[40]。右child的第一棵子樹[25]同時移成左child的最後child，因此左child[10,20]的葉是[5],[15],[25]。再欲下降[5]時，其兄弟[15]亦僅1鍵，將[5]+父10+[15]合成[5,10,15]再刪5。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

18 B 樹 解答 （續）

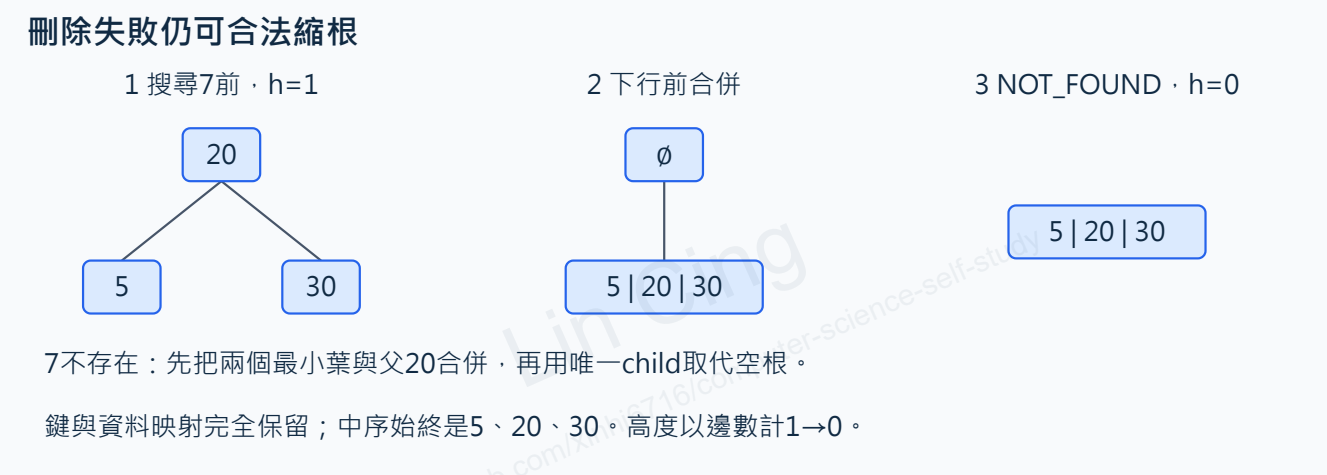
結論與檢核

最後根[30]；左內節點[20]，葉[10,15]、[25]；右內節點[40]，葉[35]、[45]。中序10,15,20,25,30,35,40,45，只少目標5，所有葉深度2。若[25]留在根30右側，便違反全域子樹範圍。

18-E05 不存在的刪除也可能改樹形

推導

搜尋7將往左child[5]，但它只有t−1=1鍵，右兄弟[30]也無多餘鍵。先與父20合成葉[5,20,30]，舊根變0鍵且僅一child。在合成葉查7仍不存在，回傳NOT_FOUND；最後以唯一child取代空內根。



結論與檢核

結果是葉根[5,20,30]，高度1降0，但總鍵集合與對應資料完全不變。刪除失敗只保證沒移除鍵，不保證結構位元不變；若API另要求失敗不寫頁，可先搜尋確認再刪，但那是額外契約且多一條搜尋路徑。

18-E06 頁內比較與I/O分開估計

推導

最壞搜尋到葉時走h條邊，兩種頁內搜尋都讀h個非根頁。線性掃描每頁最多2t−1次比較，CPU O((h+1)t)；binary search每頁O(log t)次比較，CPU O((h+1)log t)。

結論與檢核

binary search只加速找位置；在陣列型節點中，插鍵仍可能移動Θ(t)個鍵與指標，split也搬Θ(t)項。沿高h路徑可發生O(h)次split，所以插入CPU仍O(t(h+1))，而頁I/O仍O(h+1)。若硬體/節點表示改變，應另列成本模型，不混淆兩種分母。

18 B 樹 解答 （續）

原創驗算程式附錄: btree_checks.py

```
"""B-tree operations with invariant and sorted-set comparisons after every step."""
from bisect import bisect_left
from dataclasses import dataclass, field
from random import Random


@dataclass
class Node:
    keys: list = field(default_factory=list)
    children: list = field(default_factory=list)


class BTree:
    def __init__(self, degree=2):
        assert degree>=2
        self.t=degree
        self.root=Node()
        self.events=set()

    def search(self,key):
        node=self.root
        while True:
            i=bisect_left(node.keys,key)
            if i<len(node.keys) and node.keys[i]==key:
                return True
            if not node.children:
                return False
            node=node.children[i]

    def split(self,parent,i):
        t=self.t
        left=parent.children[i]
        median=left.keys[t-1]
        right=Node(left.keys[t:],left.children[t:] if left.children else [])
        left.keys=left.keys[:t-1]
        if left.children:
            left.children=left.children[:t]
        parent.keys.insert(i,median)
        parent.children.insert(i+1,right)
        self.events.add("split")

    def insert(self,key):
        if self.search(key):
            return False
        if len(self.root.keys)==2*self.t-1:
            self.root=Node([], [self.root])
            self.split(self.root,0)
            self.events.add("root-grow")
        node=self.root
        while node.children:
            i=bisect_left(node.keys,key)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

18 B 樹 解答 （續）

```
            if len(node.children[i].keys)==2*self.t-1:
                self.split(node,i)
                if key>node.keys[i]:
                    i+=1
            node=node.children[i]
        node.keys.insert(bisect_left(node.keys,key),key)
        return True

    def merge(self,parent,i):
        left,right=parent.children[i:i+2]
        left.keys.append(parent.keys.pop(i))
        left.keys.extend(right.keys)
        left.children.extend(right.children)
        parent.children.pop(i+1)
        self.events.add("merge")
        return left

    def delete(self,key):
        found=self._delete(self.root,key)
        if not self.root.keys and self.root.children:
            self.root=self.root.children[0]
            self.events.add("root-shrink")
        return found

    def _delete(self,node,key):
        i=bisect_left(node.keys,key)
        if i<len(node.keys) and node.keys[i]==key:
            if not node.children:
                node.keys.pop(i)
                self.events.add("leaf-delete")
                return True
            left,right=node.children[i:i+2]
            if len(left.keys)>=self.t:
                cursor=left
                while cursor.children:
                    cursor=cursor.children[-1]
                predecessor=cursor.keys[-1]
                self._delete(left,predecessor)
                node.keys[i]=predecessor
                self.events.add("predecessor")
            elif len(right.keys)>=self.t:
                cursor=right
                while cursor.children:
                    cursor=cursor.children[0]
                successor=cursor.keys[0]
                self._delete(right,successor)
                node.keys[i]=successor
                self.events.add("successor")
            else:
                self._delete(self.merge(node,i),key)
                self.events.add("internal-merge")
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

18 B 樹 解答 （續）

```
            return True
        if not node.children:
            return False
        child=node.children[i]
        if len(child.keys)==self.t-1:
            if i>0 and len(node.children[i-1].keys)>=self.t:
                sibling=node.children[i-1]
                child.keys.insert(0,node.keys[i-1])
                node.keys[i-1]=sibling.keys.pop()
                if sibling.children:
                    child.children.insert(0,sibling.children.pop())
                self.events.add("borrow-left")
            elif i<len(node.keys) and len(node.children[i+1].keys)>=self.t:
                sibling=node.children[i+1]
                child.keys.append(node.keys[i])
                node.keys[i]=sibling.keys.pop(0)
                if sibling.children:
                    child.children.append(sibling.children.pop(0))
                self.events.add("borrow-right")
            elif i<len(node.keys):
                child=self.merge(node,i)
            else:
                child=self.merge(node,i-1)
        return self._delete(child,key)

    def validate(self):
        leaf_depths=[]
        seen=set()
        def visit(node,low,high,depth,root=False):
            assert id(node) not in seen
            seen.add(id(node))
            assert node.keys==sorted(set(node.keys))
            assert all(low<key<high for key in node.keys)
            assert len(node.keys)<=2*self.t-1
            assert root or len(node.keys)>=self.t-1
            if not node.children:
                leaf_depths.append(depth)
                return node.keys[:]
            assert len(node.children)==len(node.keys)+1
            assert node.keys or not root
            bounds=[low]+node.keys+[high]
            result=[]
            for i,child in enumerate(node.children):
                result.extend(visit(child,bounds[i],bounds[i+1],depth+1))
                if i<len(node.keys):
                    result.append(node.keys[i])
            return result
        result=visit(self.root,float("-inf"),float("inf"),0,True)
        assert len(set(leaf_depths))==1
        return result
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

18 B 樹 解答 （續）

```
def main():
    tree=BTree()
    for key in (10,20,30,25):
        tree.insert(key)
        tree.validate()
    assert tree.root.keys==[20]
    assert [node.keys for node in tree.root.children]==[[10],[25,30]]
    borrow=BTree()
    borrow.root=Node([20],[Node([10],[Node([5]),Node([15])]),Node([30,40],
[Node([25]),Node([35]),Node([45])])])
    assert borrow.validate()==[5,10,15,20,25,30,35,40,45]
    assert borrow.delete(5)
    assert borrow.validate()==[10,15,20,25,30,35,40,45]
    assert borrow.root.keys==[30]
    assert borrow.root.children[0].keys==[20]

    events=set()
    operations=0
    for degree in (2,3,5):
        for seed in range(8):
            rng=Random(seed)
            tree=BTree(degree)
            expected=set()
            for _ in range(350):
                key=rng.randrange(120)
                if rng.random()<0.55:
                    assert tree.insert(key)==(key not in expected)
                    expected.add(key)
                else:
                    assert tree.delete(key)==(key in expected)
                    expected.discard(key)
                assert tree.validate()==sorted(expected)
                operations+=1
            for key in list(expected):
                assert tree.delete(key)
                expected.remove(key)
                assert tree.validate()==sorted(expected)
                operations+=1
            events.update(tree.events)
    required={"split","root-grow","leaf-delete","merge","root-
shrink","predecessor","successor","internal-merge","borrow-left","borrow-right"}
    assert required<=events,(required-events)
    assert 200**3-1<10**9<=200**4-1
    assert 2*100**4-1<=10**9<2*100**5-1
    print(f"PASS BTree_mixed_and_drain_operations={operations} branches={sorted(events)}
height_n1e9_t100=3_or4")


if __name__=="__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

18 B 樹 解答 （續）

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study